

Module 9 Notes: Sorted Linked List Operations

1. Current progress analysis

You are ready for Module 9 because you already understand the main linked-list ideas needed for sorted linked-list operations.

From Module 1: Arrays vs linked lists

You learned that arrays and linked lists solve different problems.

Structure	Strength	Weakness
Array	Fast direct access by index	Fixed size and shifting may be needed during insertion or deletion
Linked list	Flexible size and easier pointer-based insertion or deletion	No direct jumping to an index

This matters because Module 9 uses linked-list pointer changes instead of array shifting.

From Module 2: Node anatomy and pointer movement

You learned that a singly linked-list node has two main parts:

[data | next]

Part	Meaning
data	The value stored in the node
next	The address of the next node

C node definition:

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

You also learned the key pointer movement:

```
p = p->next;
```

This moves pointer `p` to the next node.

Module 9 depends heavily on:

```
p = p->next;  
q = p;  
p != NULL;  
p->data;  
p->next;
```

From Module 3: Traversal and display

You learned traversal.

Traversal means visiting nodes one by one from the first node until `NULL`.

Example:

```
first -> 8 -> 3 -> 7 -> 12 -> NULL
```

Basic traversal pattern:

```
while (p != NULL) {  
    // use p->data  
    p = p->next;  
}
```

This matters because `isSorted` and duplicate removal are traversal-based operations.

From Module 4: Recursion

You learned that linked-list problems often become smaller by moving from one node to the next node.

Example:

```
RFunction(p->next);
```

Module 9 is mostly iterative, but the same idea still matters: each step moves to the next smaller part of the list.

From Module 5: Count, sum, max, and min

You learned this pattern:

```
while (p != NULL) {  
    // do work at current node  
    p = p->next;  
}
```

Module 9 uses the same pattern.

Operation	Work done while traversing
isSorted	Check if the current value is smaller than the previous value
RemoveDuplicates	Compare neighboring nodes
SortedInsert	Find the correct position for a new value

From Module 6: Search using `p` and `q`

You learned that some operations need two pointers:

Pointer	Role
<code>p</code>	Current pointer
<code>q</code>	Previous or trailing pointer

Module 9 also uses `p` and `q`.

In sorted insertion:

```
q follows one node behind p
```

From Module 7: Insertion by position

You learned the safe insertion pattern:

```
t->next = p->next;  
p->next = t;
```

The order matters.

First, the new node must point to the old next node. Then, the previous node points to the new node.

This prevents losing the rest of the list.

From Module 8: Building lists and `InsertLast`

You learned how to append nodes efficiently using a `last` pointer.

Module 9 continues from that idea, but instead of always inserting at the end, we insert based on sorted order.

2. Big idea of Module 9

Module 9 is about working with a **sorted singly linked list**.

A sorted linked list has values arranged in increasing order.

Example:

```
3 -> 7 -> 9 -> 15 -> 20 -> NULL
```

This list is sorted because:

```
3 <= 7 <= 9 <= 15 <= 20
```

Module 9 teaches three main operations:

1. `SortedInsert`
2. `isSorted`
3. `RemoveDuplicates`

Operation	Purpose
SortedInsert	Insert a new node without breaking sorted order
isSorted	Check whether a list is sorted
RemoveDuplicates	Remove repeated values from a sorted list

3. What is a sorted linked list?

A sorted linked list is a linked list where each value is less than or equal to the value after it.

Example of a sorted list:

```
3 -> 5 -> 8 -> 12 -> NULL
```

This is sorted because:

```
3 <= 5 <= 8 <= 12
```

Example of an unsorted list:

```
3 -> 5 -> 2 -> 12 -> NULL
```

This is not sorted because:

```
2 < 5
```

The value 2 appears after 5, so the increasing order is broken.

4. Operation 1: Sorted insertion

What is sorted insertion?

Sorted insertion means inserting a new node into the correct position so the list remains sorted.

Example:

```
3 -> 7 -> 9 -> 15 -> 20 -> NULL
```

Insert:

```
18
```

Result:

```
3 -> 7 -> 9 -> 15 -> 18 -> 20 -> NULL
```

The new value `18` is placed between `15` and `20`.

Important idea:

```
Sorted insertion does not insert by index.  
Sorted insertion inserts by value order.
```

5. Why sorted insertion is different from normal insertion

In Module 7, insertion used an index.

Example:

```
Insert(first, 3, 10);
```

That means insert based on position.

But sorted insertion uses the value itself.

Example:

```
SortedInsert(first, 18);
```

That means insert `18` wherever it belongs in sorted order.

Normal insertion	Sorted insertion
Uses an index	Uses value comparison
Inserts at a given position	Finds the correct position automatically
May break sorted order	Keeps sorted order

6. Main idea of SortedInsert

Suppose the list is:

```
3 -> 7 -> 9 -> 15 -> 20 -> NULL
```

We want to insert:

```
18
```

We move through the list while the current node value is smaller than 18.

```
3 is smaller than 18, move
7 is smaller than 18, move
9 is smaller than 18, move
15 is smaller than 18, move
20 is not smaller than 18, stop
```

So 18 must be inserted before 20 and after 15.

Result:

```
3 -> 7 -> 9 -> 15 -> 18 -> 20 -> NULL
```

7. The two important pointers in sorted insertion

Sorted insertion usually uses two pointers:

Pointer	Role
p	Leading pointer; searches for the correct position

Pointer	Role
<code>q</code>	Trailing pointer; stays one node behind <code>p</code>

At the stopping point for inserting `18`:

```
3 -> 7 -> 9 -> 15 -> 20 -> NULL
      q      p
```

Here:

```
q points to 15
p points to 20
```

The new node should be inserted after `q` and before `p`.

8. Why do we need `q`?

When `p` reaches the first node that is not smaller than the new value, we need to insert before `p`.

But in a singly linked list, we cannot directly move backward.

So we keep `q` one node behind `p`.

Example:

```
15 -> 20
q      p
```

To insert `18`, we need to change the link of `15`.

That means we need `q`.

The insertion will be:

```
15 -> 18 -> 20
```

Without `q`, we would know where `p` is, but we would not know which previous node should point to the new node.

9. Important condition in sorted insertion

The main loop is:

```
while (p != NULL && p->data < x) {  
    q = p;  
    p = p->next;  
}
```

Meaning:

Keep moving while:
1. p is not NULL
2. p->data is smaller than the value we want to insert

Why do we check `p != NULL` first?

Because if `p` is `NULL`, this is unsafe:

```
p->data
```

So the safe condition is:

```
p != NULL && p->data < x
```

Not this:

```
p->data < x && p != NULL
```

The first condition protects the second condition.

10. SortedInsert code in C

```
void SortedInsert(struct Node *p, int x) {  
    struct Node *t;  
    struct Node *q = NULL;  
  
    t = (struct Node *)malloc(sizeof(struct Node));  
    t->data = x;
```

```

t->next = NULL;

if (first == NULL) {
    first = t;
    return;
}

while (p != NULL && p->data < x) {
    q = p;
    p = p->next;
}

if (q == NULL) {
    t->next = first;
    first = t;
} else {
    t->next = q->next;
    q->next = t;
}
}

```

11. Explanation of SortedInsert code

Function header

```
void SortedInsert(struct Node *p, int x)
```

Meaning:

Parameter	Meaning
p	Pointer used to move through the list
x	Value to insert

The function returns `void` because it directly changes the linked list.

Create pointer variables

```

struct Node *t;
struct Node *q = NULL;

```

Meaning:

Pointer	Meaning
t	New node to be inserted
q	Previous pointer, starts as NULL

At the beginning, q is NULL because there is no previous node before the first node.

Allocate the new node

```
t = (struct Node *)malloc(sizeof(struct Node));
t->data = x;
t->next = NULL;
```

Meaning:

1. Create a new node in heap memory.
2. Store x inside the node.
3. Set its next to NULL for now.

The node is not connected yet.

Empty list case

```
if (first == NULL) {
    first = t;
    return;
}
```

If the list is empty, the new node becomes the first node.

Before:

```
first = NULL
```

After inserting 10:

```
first -> 10 -> NULL
```

Then the function returns because insertion is complete.

Search for the correct position

```
while (p != NULL && p->data < x) {  
    q = p;  
    p = p->next;  
}
```

This loop moves through all nodes smaller than `x`.

At the end:

```
q points to the node before the insertion place  
p points to the node after the insertion place
```

Insert at the head

```
if (q == NULL) {  
    t->next = first;  
    first = t;  
}
```

This happens when the new value is smaller than or equal to the first node.

Example:

```
10 -> 20 -> 30 -> NULL
```

Insert `5`.

Since `5` belongs before `10`, `q` remains `NULL`.

Result:

```
5 -> 10 -> 20 -> 30 -> NULL
```

Insert in the middle or at the end

```
else {  
    t->next = q->next;  
    q->next = t;  
}
```

This inserts the new node after `q`.

The order matters:

```
t->next = q->next;  
q->next = t;
```

First, the new node points to the old next node. Then, `q` points to the new node.

12. Dry run: insert in the middle

Original list:

```
3 -> 7 -> 9 -> 15 -> 20 -> NULL
```

Insert:

```
18
```

Start:

```
p = 3  
q = NULL
```

Move while `p->data < 18`.

Step	q	p	Check
Start	NULL	3	3 < 18, move
1	3	7	7 < 18, move
2	7	9	9 < 18, move

Step	q	p	Check
3	9	15	15 < 18, move
4	15	20	20 < 18 is false, stop

Now:

```
q points to 15
p points to 20
```

Create new node:

```
18
```

Link it:

```
t->next = q->next;
q->next = t;
```

Result:

```
3 -> 7 -> 9 -> 15 -> 18 -> 20 -> NULL
```

13. Sorted insertion case 1: Empty list

Before:

```
first = NULL
```

Call:

```
SortedInsert(first, 10);
```

A new node is created:

```
10 -> NULL
```

Since the list is empty:

```
first = t;
```

Result:

```
first -> 10 -> NULL
```

Complexity:

```
O(1)
```

Only a fixed number of pointer changes are needed.

14. Sorted insertion case 2: Insert smallest value

Original list:

```
10 -> 20 -> 30 -> NULL
```

Insert:

```
5
```

At the start:

```
p = 10  
q = NULL
```

Check:

```
10 < 5 is false
```

So the loop does not move.

Since `q == NULL`, insert at the head:

```
t->next = first;  
first = t;
```

Result:

```
5 -> 10 -> 20 -> 30 -> NULL
```

15. Sorted insertion case 3: Insert in the middle

Original list:

```
10 -> 20 -> 30 -> 40 -> NULL
```

Insert:

```
25
```

The correct position is between **20** and **30**.

At stopping point:

```
10 -> 20 -> 30 -> 40 -> NULL  
      q      p
```

Insert:

```
t->next = q->next;  
q->next = t;
```

Result:

```
10 -> 20 -> 25 -> 30 -> 40 -> NULL
```

16. Sorted insertion case 4: Insert largest value

Original list:

```
10 -> 20 -> 30 -> NULL
```

Insert:

```
50
```

The loop moves until `p` becomes `NULL`.

At stopping point:

```
10 -> 20 -> 30 -> NULL
      q   p
```

Here:

```
q points to 30
p is NULL
```

Insert after `q`:

```
t->next = q->next;
q->next = t;
```

Since `q->next` is `NULL`, the new node becomes the last node.

Result:

```
10 -> 20 -> 30 -> 50 -> NULL
```

17. Important edge case: Missing smallest-value case

A common beginner mistake is writing sorted insertion that only handles insertion after `q`.

Wrong idea:

```
t->next = q->next;  
q->next = t;
```

This fails when the new value must become the first node.

Why?

If the new value is smaller than the first node, then `q` is still `NULL`.

This is unsafe:

```
q->next
```

because `q` does not point to a real node.

Correct solution:

```
if (q == NULL) {  
    t->next = first;  
    first = t;  
} else {  
    t->next = q->next;  
    q->next = t;  
}
```

18. Operation 2: Checking if a list is sorted

What does `isSorted` do?

`isSorted` checks whether the linked list is in increasing order.

It returns:

Return value	Meaning
1	The list is sorted
0	The list is not sorted

Example sorted list:

```
3 -> 5 -> 8 -> 12 -> NULL
```

Return:

```
1
```

Example unsorted list:

```
3 -> 5 -> 2 -> 12 -> NULL
```

Return:

```
0
```

19. Main idea of `isSorted`

To check if a list is sorted, compare the current value with the previous value.

Example:

```
3 -> 5 -> 8 -> 12 -> NULL
```

We check:

```
3 is okay  
5 is not smaller than 3  
8 is not smaller than 5  
12 is not smaller than 8
```

So the list is sorted.

For this list:

```
3 -> 5 -> 2 -> 12 -> NULL
```

We check:

```
3 is okay  
5 is not smaller than 3  
2 is smaller than 5
```

So the list is not sorted.

20. Why we need a previous value variable

A singly linked list cannot move backward.

So while moving forward, we remember the previous value in a variable.

Usually this variable is called `x`.

```
int x = INT32_MIN;
```

`x` stores the previous value.

At each node:

1. Compare `p->data` with `x`.
 2. If `p->data < x`, the list is not sorted.
 3. Otherwise, update `x = p->data`.
 4. Move to the next node.
-

21. Why initialize `x` with a very small value?

At the first node, there is no previous node.

So we need a starting previous value that is smaller than any normal value in the list.

That is why we use:

```
#include <stdint.h>  
  
int x = INT32_MIN;
```

`INT32_MIN` means a very small integer value.

This makes the first comparison safe.

Example:

```
First node value = 3
x = very small value
```

Check:

```
3 < very small value
```

This is false, so the first node is accepted.

22. isSorted code in C

```
#include <stdint.h>

int isSorted(struct Node *p) {
    int x = INT32_MIN;

    while (p != NULL) {
        if (p->data < x) {
            return 0;
        }

        x = p->data;
        p = p->next;
    }

    return 1;
}
```

23. Explanation of isSorted code

Start with a very small previous value

```
int x = INT32_MIN;
```

This means:

Before checking the first node, assume the previous value is extremely small.

Traverse the list

```
while (p != NULL)
```

This keeps moving until the list ends.

Check for a sorted-order break

```
if (p->data < x) {  
    return 0;  
}
```

If the current value is smaller than the previous value, the list is not sorted.

Update previous value

```
x = p->data;
```

Now the current value becomes the previous value for the next node.

Move forward

```
p = p->next;
```

Move to the next node.

Return true if no problem was found

```
return 1;
```

If the loop reaches `NULL`, no sorted-order problem was found.

So the list is sorted.

24. Dry run: `isSorted` on a sorted list

List:

```
3 -> 5 -> 8 -> 12 -> NULL
```

Start:

```
x = very small value  
p = 3
```

Step	<code>p->data</code>	Previous <code>x</code>	Check	Result
1	3	very small	<code>3 < very small</code> is false	okay
2	5	3	<code>5 < 3</code> is false	okay
3	8	5	<code>8 < 5</code> is false	okay
4	12	8	<code>12 < 8</code> is false	okay
5	<code>NULL</code>	12	stop	return <code>1</code>

Final answer:

```
1
```

The list is sorted.

25. Dry run: `isSorted` on an unsorted list

List:

```
3 -> 5 -> 2 -> 12 -> NULL
```

Start:

```
x = very small value  
p = 3
```

Step	p->data	Previous	x	Check	Result
1	3		very small	3 < very small is false	okay
2	5		3	5 < 3 is false	okay
3	2		5	2 < 5 is true	return 0

Final answer:

0

The list is not sorted.

26. Empty list and one-node list for isSorted

Empty list

An empty list is:

```
first = NULL
```

There are no values out of order.

So isSorted returns:

1

This is usually considered sorted.

One-node list

Example:

```
10 -> NULL
```


A one-node list is always sorted because there is no second value to compare with.

So `isSorted` returns:

1

27. Operation 3: Removing duplicates from a sorted linked list

What does duplicate removal do?

Duplicate removal deletes repeated values from the linked list.

Example:

3 -> 5 -> 5 -> 8 -> 8 -> 8 -> NULL

After removing duplicates:

3 -> 5 -> 8 -> NULL

Only one copy of each value remains.

28. Why duplicate removal assumes the list is sorted

In a sorted list, duplicate values are next to each other.

Example:

3 -> 5 -> 5 -> 8 -> 8 -> 8 -> NULL

The duplicate `5`s are adjacent. The duplicate `8`s are adjacent.

This makes duplicate removal easier.

We only need to compare neighboring nodes.

29. What goes wrong if the list is not sorted?

Example unsorted list:

```
5 -> 3 -> 5 -> 8 -> NULL
```

There are two **5**s, but they are not next to each other.

A duplicate-removal function that compares only neighboring nodes will not remove the second **5**.

So this duplicate removal method works properly only when the list is sorted.

30. Pointers used in duplicate removal

Duplicate removal uses two pointers:

Pointer	Role
p	Points to the current unique node
q	Points to the next node to compare

Usually:

```
q = p->next;
```

So **q** starts one node ahead of **p**.

Example:

```
3 -> 5 -> 5 -> 8 -> NULL
p   q
```

We compare:

```
p->data != q->data
```

or:

```
p->data == q->data
```

31. Main logic of duplicate removal

There are two cases.

Case 1: Values are different

Example:

```
3 -> 5  
p    q
```

Since **3** and **5** are different, both nodes should stay.

Move both pointers forward:

```
p = q;  
q = q->next;
```

Case 2: Values are equal

Example:

```
5 -> 5  
p    q
```

Since both values are equal, **q** is a duplicate.

Remove **q**:

```
p->next = q->next;  
free(q);  
q = p->next;
```

Important:

p does not move when q is deleted.

Why?

Because there may be more duplicates after q.

Example:

8 -> 8 -> 8

If p moves too early, one duplicate may be skipped.

32. RemoveDuplicates code in C

```
void RemoveDuplicates(struct Node *p) {
    struct Node *q;

    if (p == NULL) {
        return;
    }

    q = p->next;

    while (q != NULL) {
        if (p->data != q->data) {
            p = q;
            q = q->next;
        } else {
            p->next = q->next;
            free(q);
            q = p->next;
        }
    }
}
```

33. Explanation of RemoveDuplicates code

Function header

```
void RemoveDuplicates(struct Node *p)
```

The function receives a pointer to the first node.

It changes the list directly, so it returns `void`.

Declare `q`

```
struct Node *q;
```

`q` is used to compare the node after `p`.

Empty list check

```
if (p == NULL) {  
    return;  
}
```

If the list is empty, there are no duplicates to remove.

This check also prevents this unsafe line:

```
q = p->next;
```

because if `p` is `NULL`, then `p->next` is invalid.

Start `q` one node ahead

```
q = p->next;
```

Now `p` points to the current node, and `q` points to the next node.

Continue while `q` is valid

```
while (q != NULL)
```

If `q` becomes `NULL`, there is no next node to compare.

So the loop stops.

If values are different

```
if (p->data != q->data) {  
    p = q;  
    q = q->next;  
}
```

Both nodes are different, so both should remain.

Move forward.

If values are equal

```
else {  
    p->next = q->next;  
    free(q);  
    q = p->next;  
}
```

The node `q` is a duplicate.

Steps:

1. Make `p` skip over `q`.
 2. Free the duplicate node.
 3. Reset `q` to the node after `p`.
-

34. Dry run: Remove duplicates

Original list:

```
3 -> 5 -> 5 -> 8 -> 8 -> 8 -> NULL
```

Start:

```
p = 3  
q = 5
```

Step 1

```
p = 3  
q = 5
```

Compare:

```
3 != 5
```

They are different.

Move both pointers:

```
p = q;  
q = q->next;
```

Now:

```
3 -> 5 -> 5 -> 8 -> 8 -> 8 -> NULL  
p      q
```

Step 2

```
p = 5  
q = 5
```

Compare:

```
5 == 5
```

They are equal.

So **q** is a duplicate.

Before deletion:

```
3 -> 5 -> 5 -> 8 -> 8 -> 8 -> NULL
    p   q
```

Do:

```
p->next = q->next;
free(q);
q = p->next;
```

After deletion:

```
3 -> 5 -> 8 -> 8 -> 8 -> NULL
    p   q
```

Important:

```
p stays at 5
q moves to 8
```

Step 3

```
p = 5
q = 8
```

Compare:

```
5 != 8
```

They are different.

Move both:

```
3 -> 5 -> 8 -> 8 -> 8 -> NULL
      p      q
```

Step 4

```
p = 8
q = 8
```

Compare:

```
8 == 8
```

Delete `q`.

After deletion:

```
3 -> 5 -> 8 -> 8 -> NULL
      p      q
```

Step 5

```
p = 8
q = 8
```

Again:

```
8 == 8
```

Delete `q`.

After deletion:

```
3 -> 5 -> 8 -> NULL
    p    q
```

Now `q` is `NULL`, so the loop stops.

Final list:

```
3 -> 5 -> 8 -> NULL
```

35. Why `p` stays still after deleting `q`

This is one of the most important parts of duplicate removal.

Suppose we have:

```
8 -> 8 -> 8 -> NULL
p    q
```

If we delete `q`, the list becomes:

```
8 -> 8 -> NULL
p    q
```

There is still another duplicate `8` after `p`.

So `p` must stay at the first `8` and compare again.

Correct code:

```
p->next = q->next;
free(q);
q = p->next;
```

Wrong idea:

```
p = p->next;
```

That may skip a duplicate.

36. Full C program for Module 9

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *first = NULL;

void Display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}

void SortedInsert(struct Node *p, int x) {
    struct Node *t;
    struct Node *q = NULL;

    t = (struct Node *)malloc(sizeof(struct Node));
    t->data = x;
    t->next = NULL;

    if (first == NULL) {
        first = t;
        return;
    }

    while (p != NULL && p->data < x) {
        q = p;
        p = p->next;
    }

    if (q == NULL) {
        t->next = first;
        first = t;
    } else {
        t->next = q->next;
        q->next = t;
    }
}
```

```

}

int isSorted(struct Node *p) {
    int x = INT32_MIN;

    while (p != NULL) {
        if (p->data < x) {
            return 0;
        }

        x = p->data;
        p = p->next;
    }

    return 1;
}

void RemoveDuplicates(struct Node *p) {
    struct Node *q;

    if (p == NULL) {
        return;
    }

    q = p->next;

    while (q != NULL) {
        if (p->data != q->data) {
            p = q;
            q = q->next;
        } else {
            p->next = q->next;
            free(q);
            q = p->next;
        }
    }
}

int main() {
    SortedInsert(first, 10);
    SortedInsert(first, 20);
    SortedInsert(first, 30);
    SortedInsert(first, 40);
    SortedInsert(first, 25);
    SortedInsert(first, 5);
    SortedInsert(first, 30);
    SortedInsert(first, 30);
}

```

```

    printf("List after sorted insertions: ");
    Display(first);

    if (isSorted(first)) {
        printf("\nList is sorted");
    } else {
        printf("\nList is not sorted");
    }

    RemoveDuplicates(first);

    printf("\nList after removing duplicates: ");
    Display(first);

    return 0;
}

```

Expected output:

```

List after sorted insertions: 5 10 20 25 30 30 30 40
List is sorted
List after removing duplicates: 5 10 20 25 30 40

```

37. C++ memory note

In C, nodes are created using:

```
malloc
```

and removed using:

```
free
```

In C++, nodes are usually created using:

```
new
```

and removed using:

delete

Important rule:

Language	Create node	Delete node
C	<code>malloc</code>	<code>free</code>
C++	<code>new</code>	<code>delete</code>

Do not mix them.

Wrong:

```
Node *p = new Node;
free(p); // wrong
```

Wrong:

```
struct Node *p = malloc(sizeof(struct Node));
delete p; // wrong
```

38. Complexity analysis

Let `n` be the number of nodes in the linked list.

SortedInsert complexity

Best case

The best case happens when:

1. The list is empty.
2. The new value belongs at the head.

Example:

```
10 -> 20 -> 30 -> NULL
```

Insert `5`.

The function does not need to traverse far.

Time complexity:

$O(1)$

Extra space:

$O(1)$

Worst case

The worst case happens when the new value belongs at the end.

Example:

10 -> 20 -> 30 -> 40 -> NULL

Insert 50.

The function must visit every node before inserting.

Time complexity:

$O(n)$

Extra space:

$O(1)$

isSorted complexity

Best case

The best case happens when the list is unsorted near the beginning.

Example:

```
10 -> 5 -> 20 -> NULL
```

The function quickly finds:

```
5 < 10
```

Time complexity:

```
O(1)
```

Worst case

The worst case happens when:

1. The list is fully sorted.
2. The list becomes unsorted only near the end.

Example:

```
3 -> 5 -> 8 -> 12 -> NULL
```

The function must check all nodes.

Time complexity:

```
O(n)
```

Extra space:

```
O(1)
```

RemoveDuplicates complexity

Duplicate removal visits the list from beginning to end.

Even if there are duplicates, each node is processed once.

Time complexity:

$O(n)$

Extra space:

$O(1)$

It uses only a few pointer variables, mainly `p` and `q`.

39. Complexity comparison table

Operation	Best-case time	Worst-case time	Extra space	Reason
<code>SortedInsert</code>	$O(1)$	$O(n)$	$O(1)$	May insert at head or may need traversal to the end
<code>isSorted</code>	$O(1)$	$O(n)$	$O(1)$	May find disorder early or may check the whole list
<code>RemoveDuplicates</code>	$O(n)$	$O(n)$	$O(1)$	Traverses through the list and removes adjacent duplicates

40. Common beginner mistakes

Mistake 1: Forgetting the empty list case

Wrong idea:

```
while (p != NULL && p->data < x) {  
    q = p;  
    p = p->next;  
}
```

Then directly using `q`.

Problem:

If the list is empty, `first == NULL`.

So we must handle it:

```
if (first == NULL) {  
    first = t;  
    return;  
}
```

Mistake 2: Forgetting the smallest-value case

If the new value belongs before the first node, `q` remains `NULL`.

Unsafe code:

```
q->next = t;
```

Correct code:

```
if (q == NULL) {  
    t->next = first;  
    first = t;  
}
```

Mistake 3: Writing the link changes in the wrong order

Wrong:

```
q->next = t;  
t->next = q->next;
```

Problem:

After `q->next = t`, the original next node may be lost.

Correct:

```
t->next = q->next;  
q->next = t;
```

Rule:

Copy the old link before overwriting it.

Mistake 4: Using duplicate removal on an unsorted list

Example:

5 -> 3 -> 5 -> 8 -> NULL

The duplicate 5s are not adjacent.

The normal duplicate-removal function compares neighboring nodes only.

So it may not remove all duplicates from an unsorted list.

Correct idea:

Use this duplicate removal method on sorted lists.

Mistake 5: Moving p after deleting q

Wrong idea:

```
p->next = q->next;  
free(q);  
p = p->next;
```

Problem:

This may skip a duplicate when duplicates are consecutive.

Correct:

```
p->next = q->next;  
free(q);  
q = p->next;
```

`p` stays where it is.

Mistake 6: Forgetting to check `p == NULL` before `q = p->next`

Wrong:

```
q = p->next;
```

If `p` is `NULL`, this is invalid.

Correct:

```
if (p == NULL) {  
    return;  
}  
  
q = p->next;
```

Mistake 7: Thinking reverse display and duplicate removal change the same thing

Reverse display only changes output order.

Duplicate removal actually changes links and frees nodes.

So duplicate removal changes the actual list.

41. Important pointer patterns to memorize

Pattern 1: Move `p` forward

```
p = p->next;
```

Meaning:

```
Move p to the next node.
```

Pattern 2: Move `q` behind `p`

```
q = p;  
p = p->next;
```

Meaning:

`q` becomes the previous node.
`p` moves to the next node.

Used in sorted insertion.

Pattern 3: Insert after `q`

```
t->next = q->next;  
q->next = t;
```

Meaning:

New node `t` is inserted after `q`.

Pattern 4: Insert before first

```
t->next = first;  
first = t;
```

Meaning:

New node `t` becomes the first node.

Pattern 5: Delete duplicate node `q`

```
p->next = q->next;  
free(q);  
q = p->next;
```

Meaning:

```
p skips q.  
q is freed.  
q is reset to the next node after p.
```

42. Module 9 final summary

Module 9 teaches sorted linked-list operations.

The three main operations are:

```
SortedInsert  
isSorted  
RemoveDuplicates
```

SortedInsert

Purpose:

```
Insert a new value while keeping the list sorted.
```

Main ideas:

```
Use p to search.  
Use q behind p.  
Stop when p is NULL or p->data is not smaller than x.  
Handle empty list.  
Handle new-head case.  
Insert using safe link order.
```

Key code:

```
while (p != NULL && p->data < x) {  
    q = p;  
    p = p->next;  
}
```

```
t->next = q->next;  
q->next = t;
```

isSorted

Purpose:

Check whether the list is sorted.

Main ideas:

Store previous value in x.
Compare current node data with x.
If current value is smaller than previous value, return 0.
If the whole list is checked, return 1.

Key code:

```
if (p->data < x) {  
    return 0;  
}  
  
x = p->data;  
p = p->next;
```

RemoveDuplicates

Purpose:

Remove repeated adjacent values from a sorted linked list.

Main ideas:

Duplicates are adjacent in a sorted list.
Use p and q.
If values are different, move both forward.

If values are equal, unlink and free q.
Keep p still after deleting q.

Key code:

```
if (p->data != q->data) {  
    p = q;  
    q = q->next;  
} else {  
    p->next = q->next;  
    free(q);  
    q = p->next;  
}
```

43. Mini practice questions

Practice 1

Given:

10 -> 20 -> 30 -> NULL

After `SortedInsert(first, 25)`, what is the list?

Answer:

10 -> 20 -> 25 -> 30 -> NULL

Practice 2

Given:

10 -> 20 -> 30 -> NULL

After `SortedInsert(first, 5)`, what is the list?

Answer:


```
5 -> 10 -> 20 -> 30 -> NULL
```

Practice 3

Given:

```
10 -> 20 -> 30 -> NULL
```

After `SortedInsert(first, 50)`, what is the list?

Answer:

```
10 -> 20 -> 30 -> 50 -> NULL
```

Practice 4

What does `isSorted` return for this list?

```
3 -> 5 -> 8 -> 12 -> NULL
```

Answer:

```
1
```

The list is sorted.

Practice 5

What does `isSorted` return for this list?

```
3 -> 5 -> 2 -> 12 -> NULL
```

Answer:

0

The list is not sorted because `2 < 5`.

Practice 6

Remove duplicates from:

`3 -> 5 -> 5 -> 8 -> 8 -> 8 -> NULL`

Answer:

`3 -> 5 -> 8 -> NULL`

Practice 7

Why does duplicate removal work best on sorted lists?

Answer:

Because duplicate values are next to each other in a sorted list.
So the function only needs to compare neighboring nodes.

Practice 8

Why should `p` not move immediately after deleting duplicate `q`?

Answer:

Because there may be another duplicate after `q`.
Keeping `p` still allows the function to compare `p` with the next node again.

44. Final memory checklist

Before moving to Module 10, make sure you can explain these without looking:

1. What a sorted linked list is.
2. Why sorted insertion uses value comparison instead of index.
3. The roles of `p`, `q`, and `t` in `SortedInsert`.
4. Why `q == NULL` means insert at the head.
5. Why `t->next = q->next` must happen before `q->next = t`.
6. How `isSorted` uses the previous value `x`.
7. Why `INT32_MIN` is used in `isSorted`.
8. Why duplicates are adjacent in a sorted list.
9. Why duplicate removal uses `p` and `q`.
10. Why `p` stays still when `q` is deleted.
11. The time complexity of `SortedInsert`, `isSorted`, and `RemoveDuplicates`.

45. One-page quick revision

Sorted linked list:
Values are in increasing order.
Example: 3 -> 7 -> 9 -> 15 -> NULL

SortedInsert:
Insert `x` in its correct sorted position.
Use `p` to search and `q` behind `p`.

```
while (p != NULL && p->data < x) {  
    q = p;  
    p = p->next;  
}
```

If `q == NULL`:
Insert at head.

```
t->next = first;  
first = t;
```

Otherwise:
Insert after q.

```
t->next = q->next;  
q->next = t;
```

isSorted:
Remember previous value in x.
If current value is smaller than previous value, return 0.
Otherwise return 1 after reaching NULL.

```
if (p->data < x) {  
    return 0;  
}  
x = p->data;  
p = p->next;
```

RemoveDuplicates:
Works on sorted lists because duplicates are adjacent.
Use p and q.
If equal, delete q.
If different, move both forward.

```
if (p->data != q->data) {  
    p = q;  
    q = q->next;  
} else {  
    p->next = q->next;  
    free(q);  
    q = p->next;  
}
```

Complexities:
SortedInsert: best $O(1)$, worst $O(n)$
isSorted: best $O(1)$, worst $O(n)$
RemoveDuplicates: $O(n)$
Extra space for all: $O(1)$